10 Classes and Objects 3

Objects Shopping Cart (NOTE! This is only a *small OOP exercise*, not a real shopping cart implementation)

Suppose that we want to create a simple shopping cart.
* Products can be added to a shopping cart and removed from the shopping cart.
* Each product has product number, name and price.

INSTRUCTION

* Write classes **Product**, **Item**, and **ShoppingCart**.

* The **Product** class has three fields, a parametric constructor, and getters and setters for all fields.

* The **ShoppingCart** class and the **Item** class should be as below.

ShoppingCart
- ArrayList<Item> itemList
+ ShoppingCart() + void add(Product product, int quantity) + void remove(Product product) + double getTotalPrice() + String toString()

Item
- Product product - int quantity
+ Item (Product product, int quantity) + Product getProduct() + int getQuantity() + double getSubtotal()

NOTE! Use the ready-made program class below to test your own classes. You should not modify the program class at all.

```java
public class ObjectsShoppingCartProgram {

    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();
        Product product_1 = new Product(10, "Bicycle", 500.00);
        Product product_2 = new Product(11, "Energy bar", 1.50);
        Product product_3 = new Product(12, "Water bottle", 6.00);

        System.out.println(cart);

        cart.add(product_1, 1);
        cart.add(product_2, 5);
        cart.add(product_3, 2);
        System.out.println(cart);

        cart.remove(product_2);
        System.out.println(cart);
    }
}
```

Example output:

```
There are no items in the shopping cart.

=== Shopping cart ===
10: Bicycle, quantity: 1, unit price: 500,00, subtotal: 500,00
11: Energy bar, quantity: 5, unit price: 1,50, subtotal: 7,50
12: Water bottle, quantity: 2, unit price: 6,00, subtotal: 12,00
TOTAL PRICE: 519,50 euros

=== Shopping cart ===
10: Bicycle, quantity: 1, unit price: 500,00, subtotal: 500,00
12: Water bottle, quantity: 2, unit price: 6,00, subtotal: 12,00
TOTAL PRICE: 512,00 euros
```

MathUtility

First, write a public class called **MathUtility**. This is a ***method library class*** that should have only one method, a ***static*** method named **createFiboArray**. The method should create an array of integers, populate the array with Fibonacci numbers, and return a reference to the array.

> *Fibonacci numbers* form a sequence where each number is the sum of the two preceding ones, starting always from 0 and 1.

The **createFiboArray** method should take the desired array size as its input parameter. **Note**: If the desired array size is less than two, the method should still create an array of two integers.

Then, write a program class called **MethodLibraryExercise**. The program should input the desired array size from the user, call the *createFiboArray* method of the *MathUtility* class to get an array that contains the Fibonacci numbers, and then print the numbers in the array on a single line.

IMPORTANT: In this exercise, you should ***submit only the code of the MathUtility class to Viope***. In Viope, there is a test program that will call the **createFiboArray** method of your **MathUtility** class.

Example output:

```
How many fibonacci numbers? 10
0 1 1 2 3 5 8 13 21 34
```

Objects Sort Word Pairs (Using interfaces)

Write a program called **ObjectsSortWordPairs** that first inputs pairs of English and Finnish words from the user until the user enters "OK" as the English word.

Next, the program should sort the list of WordPairs in *alphabetical order by English word* and print the words as shown in the example output.

Finally, the program should sort the list of WordPairs in *alphabetical order by Finnish word* and print the words as shown in the example output.

Example output:

```
Enter an English word (quit ends): yellow
Enter a Finnish word: keltainen

Enter an English word (quit ends): brown
Enter a Finnish word: ruskea

Enter an English word (quit ends): red
Enter a Finnish word: punainen

Enter an English word (quit ends): quit

brown = ruskea
red = punainen
yellow = keltainen

keltainen = yellow
punainen = red
ruskea = brown
```

INSTRUCTION (To be followed carefully!)

Reuse the **WordPair** class from a previous exercise.

- This time, you have to add something to the WordPair class to make WordPair objects ***comparable***. Please study the course materials to see how you can sort objects.
- After the modification, the WordPair class should still work properly with all the other programs that use it.
- Create exactly one list of *WordPairs*. You should not create any other arrays or lists.
- You should not use lambda expressions in this exercise.

Hint: To speed up coding, you can also copy/paste some code from your *ObjectsTranslationArrayList* program.

Objects Sort Word Pairs Lambda (using lambda expressions)

First, create a copy of your **ObjectsSortWordPairs** program and name the copy as **ObjectsSortWordPairsLambda**. Then, modify the copy so that you sort the list of WordPairs with ***lambda expressions***.

Example output:

```
Enter an English word (quit ends): yellow
Enter a Finnish word: keltainen

Enter an English word (quit ends): brown
Enter a Finnish word: ruskea

Enter an English word (quit ends): red
Enter a Finnish word: punainen

Enter an English word (quit ends): quit

brown = ruskea
red = punainen
yellow = keltainen

keltainen = yellow
punainen = red
ruskea = brown
```